Reviewer Pack — Minimal Quotient Space Rank and Frege Proof Length Inequalit…

Entry 859d4d2fc764 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 23:47:18 UTC

Minimal Quotient Space Rank and Frege Proof Length Inequality

Entry ID: `859d4d2fc764` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 23:47:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Quotient Spaces) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every Boolean satisfiability instance with m clauses and n variables, the minimal rank of its associated quotient space is linearly correlated with its Frege proof length, such that min_rank(Q) = Θ(λ(n, m)), where λ(n, m) is a function of n and m.

Rationale (proposer's reasoning):

> Quotient spaces provide a geometric representation of equivalence classes in an algebraic structure. In the context of Boolean satisfiability, the quotient space may capture the essence of variable interdependencies, which could be related to the complexity of proving their satisfiability. A higher minimal rank might indicate more complex interactions that lead to longer Frege proofs.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `03285a3236e5acd1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between min_rank(Q) and λ(n, m) across all 30 seeds is ≥ 0.8, with no seed producing a Frege proof length > 50 steps.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_instance(n, m):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [f'-{v}' if v.startswith('-') else f'-{v}' for v in clause]
            clauses.append(clause)
        return variables, clauses

    def homology_algorithm(variables, clauses):
        # Simplified homology algorithm to compute quotient space rank
        n = len(variables)
        M = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for var in clause:
                if var.startswith('-'):
                    row, col = int(var[1:]) - 1, n
                else:
                    row, col = int(var) - 1, n
                M[row][col] += 1
        rank = gaussian_elimination(M)
        return rank

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            # Find pivot row
            max_row = i
            for r in range(i + 1, rows):
                if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                    max_row = r
```

```
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            # Eliminate non-pivot elements
            pivot = matrix[i][i]
            for j in range(i + 1, cols):
                matrix[i][j] /= pivot
            matrix[i][i] = 1

            for r in range(rows):
                if r != i:
                    factor = matrix[r][i]
                    for j in range(i, cols):
                        matrix[r][j] -= factor * matrix[i][j]

        rank = sum(1 for row in matrix if any(row[j] != 0 for j in range(cols)))
        return rank

    def frege_proof_length(variables, clauses):
        # Simplified Frege proof length calculation
        return len(clauses) + len(variables)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        m = random.randint(1, n * (n - 1) // 2)
        variables, clauses = generate_boolean_instance(n, m)
        min_rank_Q = homology_algorithm(variables, clauses)
        proof_length = frege_proof_length(variables, clauses)
        results.append((min_rank_Q, proof_length))

    if not results:
        return {
            "metric_name": "min_rank(Q)",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    min_rank_Q_values = [r[0] for r in results]
    proof_length_values = [r[1] for r in results]

    mean_min_rank_Q = sum(min_rank_Q_values) / len(min_rank_Q_values)
    mean_proof_length = sum(proof_length_values) / len(proof_length_values)
    std_deviation = math.sqrt(sum((x - mean_min_rank_Q) ** 2 for x in min_rank_Q_values) / len(mir

    correlation_coefficient = sum((min_rank_Q_values[i] - mean_min_rank_Q) * (proof_length_values[

    return {
        "metric_name": "min_rank(Q)",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.8 and all(p <= 50 for p in proof_length_v
        "counterexample": "" if correlation_coefficient >= 0.8 and all(p <= 50 for p in proof_leng
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m
        results.append(trial_result)

    mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
    std_deviation = math.sqrt(sum((r['metric_value'] - mean_metric_value) ** 2 for r in results) /
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_deviation} support_fraction={s
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_deviation} support_fraction={s
    else:
        first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[next(i for i, r in enumerate(results)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26145f79.py", line 97, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26145f79.py", line 64, in run_trial
    variables, clauses = generate_boolean_instance(n, m)
                                                      ^
NameError: name 'm' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the conjecture's support conditions. | next: Review and fix the error in the test code to ensure it runs successfully and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14027
2	propose	ollama_remote	glm4:latest	0	0	18383
3	preregistration	ollama_remote	glm4:latest	0	0	8869
4	novelty	ollama_remote	glm4:latest	0	0	8251
5	novelty	ollama_remote	glm4:latest	0	0	8526
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50188
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12549
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11229
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17561
10	judge	ollama_remote	glm4:latest	0	0	42266

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 191849 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/859d4d2fc764.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/859d4d2fc764.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/859d4d2fc764.tar.gz` (if generated)